Graph Reconstruction

Manuale operativo e riferimento tecnico

Progetto Borsa di Ricerca

Versione documentale: 27 ottobre 2026

Indice

I Guida operativa **1**

1 Flusso operativo **2**

1.1 Finalità e ambito 2

1.2 Procedura operativa 3

2 Installazione e ambiente Conda **4**

2.1 Architettura degli ambienti 4

2.2 Requisiti hardware indicativi 5

3 Dati e configurazione **6**

3.1 Organizzazione di codice e dati 6

3.2 Configurazione locale 7

3.3 Verifica della configurazione 8

4 Esecuzione, verifica e consegna **9**

4.1 Mappa della configurazione 9

4.2 Elaborazione di un singolo nodo 10

4.3 Output prodotti 10

4.4 Verifica del risultato 12

4.5 Esecuzione batch e preparazione della consegna 12

II Architettura e riferimento tecnico **15**

5 Repository ed entrypoint **16**

5.1 Punto di ingresso della ricostruzione 16

5.2 Struttura del repository 16

5.3 Strumenti successivi alla ricostruzione 17

5.4 Caricamento della configurazione 17

6 Flusso di elaborazione nel codice **18**

6.1 Orchestrazione e sequenza generale . . . 18

6.2 Detection e riconciliazione delle osservazioni . . . 18

6.3 Geometria VGGT e costruzione degli indizi . . . 20

6.4 Costruzione, completamento e annotazione del grafo . . . 20

7 Modello dati e mappa del codice **22**

7.1 Rappresentazioni principali . . . 22

7.2 Moduli da conoscere . . . 23

7.3 Limiti implementativi noti . . . 23

8 Diagnosi e verifica delle modifiche **24**

8.1 Diagnosi di un nodo . . . 24

8.2 Verifica delle modifiche . . . 25

8.3 Strumenti specialistici . . . 25

A Riferimento operativo **26**

A.1 Ambiente e verifica . . . 26

A.2 Comandi principali . . . 26

A.3 Percorsi e artefatti . . . 27

A.4 Variabili di `.env` . . . 27

A.5 Verifica minima del risultato . . . 28

Parte I

Guida operativa

Capitolo 1

Flusso operativo

1.1 Finalità e ambito

La pipeline ricostruisce un grafo a partire da fotografie multi-vista dello stesso nodo idraulico. Individua i simboli Shurikode, associa le detection riferite allo stesso oggetto fisico e, quando disponibile e abilitata, usa la geometria prodotta da VGGT come supporto per la riconciliazione delle istanze e la stima delle connessioni. Il risultato è un grafo JSON corredato da artefatti per il controllo e la valutazione.



Figura 1.1: Il percorso di una lavorazione, dalle fotografie ai file da controllare o consegnare.

La Parte I descrive l'utilizzo operativo, dalla preparazione dell'ambiente alla consegna. La Parte II presenta gli entrypoint, il flusso applicativo e gli oggetti Python che implementano le singole fasi.

Terminologia

Nel repository la parola *nodo* compare con due significati. Un *nodo del dataset* è una scena completa, per esempio il nodo `7`, fotografata in più viste. Un *nodo del grafo* è invece uno dei componenti fisici ricostruiti all'interno di quella scena. La distinzione diventa naturale guardando i nomi dei file:

```
node7_0001.jpg
node7_0002.jpg
node7_0003.jpg
```

Queste immagini formano una sola lavorazione, identificata da `7`. Nel file finale possono invece comparire, per esempio, i componenti `15#1`, `95#1` e `116#1`. Sono gli identificativi delle istanze fisiche stimate dalla ReID, non nomi di immagini.

Gli ID della lavorazione possono contenere underscore. I file `node15_1_0001.jpg` e `node15_1_0002.jpg`, per esempio, appartengono entrambi al nodo `15_1`.

Nel seguito, il termine *lavorazione* indica il gruppo di immagini identificato da uno stesso ID, mentre *nodo* indica normalmente un componente del grafo.

1.2 Procedura operativa

Una normale esecuzione segue questo percorso:

1. si attiva l'ambiente Conda del progetto;
2. si indica in `.env` dove si trovano immagini, modello e directory di lavoro;
3. si esegue inizialmente un solo nodo, per esempio `python run.py 7`;
4. si apre il risultato sotto `<WORK_DIR>/multiview/node7/`;
5. si genera il render del grafo;
6. si esegue la valutazione, se è disponibile il ground truth;
7. si esporta il GeoJSON per QGIS, se richiesto.

Risultati attesi

Se `WORK_DIR` conserva il valore predefinito `./output`, un run del nodo `7` termina nella cartella:

`output/multiview/node7/`

Il risultato principale è `fused_graph.json`, che contiene componenti e archi. `scene_nodes.json` collega le istanze fisiche alle detection originali; `bird_view.png` consente un primo controllo della disposizione spaziale. L'insieme degli output è descritto nel Capitolo 4.

Criterio di completamento. La sola presenza della cartella non certifica il risultato. Per considerare conclusa una lavorazione devono esistere `fused_graph.json` e, dopo l'esecuzione di `python visualize.py <id>`, un render leggibile. Il riepilogo finale nel terminale deve inoltre essere coerente con il numero di viste elaborate e con i componenti attesi o osservabili.

Capitolo 2

Installazione e ambiente Conda

2.1 Architettura degli ambienti

Gli entrypoint nella root, come `run.py` ed `evaluate.py`, devono essere eseguiti nell'ambiente Conda principale `graph_rec`, descritto da `environment.yml`. L'interprete usato per VGGT è indicato da `VGGT_EMBEDDED_PYTHON` e può coincidere con quello di `graph_rec`; se la variabile non è impostata viene usato l'interprete dell'ambiente attivo.

La pipeline si avvia quindi dal normale ambiente Conda. Un runtime separato, quando necessario, va indicato esplicitamente nella configurazione locale.

> **Due file diversi.** `environment.yml` descrive Python e pacchetti dell'ambiente Conda; `.env` contiene invece percorsi e opzioni della pipeline.

Creazione dell'ambiente

Aprire un terminale nel quale il comando `conda` sia disponibile, spostarsi nella root del repository e creare l'ambiente dichiarato dal progetto:

```
conda env create -f environment.yml
conda activate graph_rec
```

Per riallineare un ambiente già esistente al file versionato:

```
conda env update -n graph_rec -f environment.yml --prune
```

L'opzione `--prune` rimuove i pacchetti non più dichiarati nel file.

> **Ambiente validato.** `environment.yml` fissa Python e lo stack Linux/CUDA usato nel benchmark. Le build di PyTorch sono prelevate dall'indice ufficiale CUDA 12.6 di PyTorch, dichiarato nello stesso file.

Installazione del decoder Shurikode

Il decoder, non incluso in `requirements.txt`, è disponibile nel repository GitHub Shurikode Context. Dopo avere attivato `graph_rec`, clonarlo e installare il pacchetto dalla sorgente:

```
git clone https://github.com/Carraro-Riccardo/Shurikode-Context.git
```

```
cd Shurikode-Context
python -m pip install .
```

I tre checkpoint dei modelli sono inclusi nel pacchetto e non richiedono un download separato.

L'import atteso dal codice è:

```
from shurikodecontext import Dec
```

Se questo import non funziona, la pipeline non riuscirà a decodificare i simboli anche se YOLO è installato correttamente.

Installazione e runtime VGGT

VGGT va installato dalla sorgente ufficiale Facebook Research. Dopo avere attivato `graph_rec`, installare la revisione usata nello sviluppo:

```
python -m pip install "git+https://github.com/facebookresearch/vggt.
    git@44b3afbd1869d8bde4894dd8ea1e293112dd5eba"
```

Il commit indicato nel comando rende riproducibile la versione di VGGT usata dal progetto.

Al primo utilizzo, i pesi del modello configurato vengono scaricati automaticamente e salvati nella cache locale. La prima esecuzione può quindi richiedere una connessione Internet e risultare sensibilmente più lenta; le esecuzioni successive riutilizzano i file già scaricati.

Nelle sessioni successive è sufficiente eseguire `conda activate graph_rec` dalla directory del repository.

2.2 Requisiti hardware indicativi

Il dimensionamento è stato ricavato da un'esecuzione completa senza cache sul nodo `node9`, composto da sette viste. La tabella riporta sia i picchi misurati sia la configurazione consigliata con un margine operativo.

Tabella 2.1: Sintesi del benchmark hardware

Voce	Valore
Scenario di prova	Cold run della pipeline su `node9`, con 7 viste
GPU di prova	NVIDIA RTX A5000 con 24 GiB di VRAM
Tempo di esecuzione	81,9 secondi
Picco RAM del processo	11,2 GiB
Picco VRAM misurato	10,85 GiB
VRAM con margine del 20%	13,02 GiB
GPU consigliata	GPU NVIDIA compatibile con CUDA, con almeno 16 GiB di VRAM
RAM di sistema	16 GiB minimi; 32 GiB consigliati
Stack validato	Python 3.10.21, PyTorch 2.6.0+cu126 e CUDA 12.6

I valori non costituiscono requisiti assoluti: il consumo di memoria e i tempi possono variare con il numero di viste, la risoluzione delle immagini, il modello e la configurazione software utilizzata.

Capitolo 3

Dati e configurazione

3.1 Organizzazione di codice e dati

La directory `data/` contiene la struttura attesa dalla pipeline. I grafi annotati sono versionati; le directory delle immagini e delle label sono predisposte ma vuote. Il checkpoint YOLO è conservato in `models/`. La struttura iniziale è:

```
data/
  images/
  labels_extended/
  graph_annotations/
```

Le immagini e le label possono essere copiate nelle directory predisposte. Per dataset aziendali o più lavorazioni indipendenti è comunque possibile usare percorsi esterni, indicandoli nel file `.env`.

Convenzione di denominazione delle immagini

La discovery non visita sottocartelle e legge soltanto file con estensione minuscola `.jpg` presenti direttamente in `IMAGES_DIR`. Il nome deve rispettare la forma:

`node<id>_<frame_numerico>.jpg`

L'ultima parte prima dell'estensione è sempre il numero del frame. Tutto ciò che si trova tra `node` e l'ultimo underscore è l'ID della lavorazione. Sono quindi validi entrambi questi gruppi:

```
node7_0001.jpg
node7_0002.jpg

node15_1_0001.jpg
node15_1_0002.jpg
```

File come `node7_frontale.jpg`, immagini PNG o estensioni `.JPG` non vengono rilevati. Se un nodo non viene trovato, verificare prima nomi ed estensioni.

3.2 Configurazione locale

Nella root del progetto è presente `.env.template`. Copiarlo in `.env`; il nuovo file resta locale e non deve contenere dati da versionare:

```
Copy-Item .env.template .env
```

Per usare la struttura predisposta nel repository, la parte principale del file diventa:

```
IMAGES_DIR=./data/images
LABELS_EXT_DIR=./data/labels_extended
YOLO_MODEL_PATH=./models/yolo_detector.pt

ASSETS_DIR=./graph_reconstruction/assets
ANNOTATIONS_DIR=./data/graph_annotations
WORK_DIR=./output
```

I valori mostrati coincidono con i percorsi predefiniti del repository e vanno modificati soltanto quando input o modello si trovano altrove. `IMAGES_DIR` e `YOLO_MODEL_PATH` servono alla ricostruzione; `LABELS_EXT_DIR` è usata soprattutto da `evaluate.py`. In assenza di label può indicare una directory vuota; la valutazione supervisionata verrà omessa.

`WORK_DIR` è la directory radice degli output: la pipeline vi crea le cache delle detection, i risultati multivista, i confronti e gli eventuali file diagnostici. `ASSETS_DIR` e `ANNOTATIONS_DIR` indicano invece le risorse di supporto descritte di seguito.

I percorsi relativi, come `./graph_reconstruction/assets`, sono risolti rispetto alla root del repository, non rispetto alla directory dalla quale è stato aperto il terminale. Per dataset e modelli esterni sono preferibili percorsi assoluti.

Se una variabile omonima esiste nell'ambiente di Windows o nella sessione Conda, ha precedenza sul valore scritto in `.env`. In caso di percorsi inattesi, verificare sia i valori caricati sia le variabili della sessione.

Dati di riferimento

La directory indicata da `ASSETS_DIR` contiene mapping, porte e primitive SVG dei componenti Shurikode. Queste risorse servono a rappresentare correttamente i componenti nelle visualizzazioni CAD e QGIS, ma non modificano la topologia del grafo ricostruito. Le primitive attuali sono risorse di progetto e potranno essere aggiornate o adattate agli standard aziendali.

La directory indicata da `ANNOTATIONS_DIR` contiene i grafi annotati con nome `node<id>_gt.json`. Questi file non servono a produrre `fused_graph.json`; servono a confrontare il risultato con un grafo atteso. Anche i grafi campione sotto `evaluation/standard_comparison/` appartengono alla fase di valutazione, non alla ricostruzione.

Formato delle annotazioni di grafo

Ogni file di annotazione descrive i componenti di un nodo idraulico e le connessioni attese tra essi. Il formato è il seguente:

```
{
  "node_id":"11", "best_view":"node11_0003.jpg",
  "nodes": [
    {"nid":"95#1","code":95,"name":"Bout bic.","valence":2,"cx":0.58,"cy":0.31},
    {"nid":"15#2","code":15,"name":"Saracinesca","valence":2,"cx":0.57,"cy":0.34}
  ],
  "edges": [["95#1", "15#2"]]
}
```

`node_id` identifica il gruppo di immagini e `best_view` la vista usata come riferimento per l'annotazione. In `nodes`, `nid` è l'identificatore univoco nel formato `<code>#<instance_id>`; `code` è il codice Shurikode, `valence` il numero atteso di connessioni e `cx`, `cy` le coordinate normalizzate del centro. `edges` contiene infine le coppie di `nid` collegate nel grafo atteso.

Opzioni VGGT e debug

Per il primo run si raccomanda di mantenere i valori VGGT del template. In particolare, `VGGT_EDGE_ENABLED=true` abilita la geometria 3D e `VGGT_GENERATE_IF_MISSING=true` consente di generare gli artefatti mancanti. La prima elaborazione può quindi essere sensibilmente più lenta delle successive.

Il debug dettagliato va inizialmente lasciato spento:

```
DEBUG_OUTPUT_ENABLED=false
DEBUG_OUTPUT_DIR=./output/debug
```

Il debug va abilitato soltanto per l'analisi di nodi problematici.

3.3 Verifica della configurazione

Con l'ambiente Conda attivo, si può verificare che la configurazione sia caricabile senza avviare YOLO:

```
python -c "from graph_reconstruction.config import IMAGES_DIR, WORK_DIR; print(
    IMAGES_DIR); print(WORK_DIR)"
```

I due percorsi stampati devono corrispondere alla lavorazione corrente. Se il comando segnala `Missing required path setting`, il file `.env` non esiste, non è nella root corretta oppure manca una delle variabili obbligatorie.

Capitolo 4

Esecuzione, verifica e consegna

4.1 Mappa della configurazione

Prima dell'esecuzione è utile distinguere le variabili di ambiente dai parametri globali del progetto. Il file `.env` locale contiene i valori specifici della macchina e della lavorazione corrente; non deve necessariamente ripetere tutte le opzioni facoltative presenti in `.env.template`. Quando una di queste opzioni è assente, viene usato il valore predefinito dichiarato in `graph_reconstruction/config.py`.

Elemento	Tipo o origine	Descrizione
`IMAGES_DIR`, `LABELS_EXT_DIR`, `YOLO_MODEL_PATH`	Variabili di ambiente in `.env`	Percorsi degli input, delle label estese e del checkpoint YOLO. Sono richiesti solo per sostituire i percorsi predefiniti del repository.
`WORK_DIR`, `ASSETS_DIR`, `ANNOTATIONS_DIR`	Variabili di ambiente in `.env`	Directory degli output, delle risorse grafiche e delle annotazioni di grafo.
`DEBUG_OUTPUT_ENABLED`, `DEBUG_OUTPUT_DIR`	Variabili di ambiente in `.env`	Abilitazione e destinazione dei file diagnostici facoltativi.
Opzioni VGGT elencate in `.env.template`	Variabili di ambiente facoltative	Se non sono riportate nel `.env` locale, la pipeline usa i valori predefiniti di `config.py`, compreso il percorso dell'interprete VGGT.
`CONF_THRESHOLD`, `YOLO_CONF`, `YOLO_IOU`, `YOLO_IMGSZ`	Parametri globali in `config.py`	Soglie e dimensione di inferenza; non vengono lette dal file `.env`.
`SKIP_NODES`, `VGGT_SKIP_NODES`, `MIN_IMAGES`	Parametri globali in `config.py`	Filtri applicati rispettivamente al batch, all'elaborazione VGGT e al numero minimo di viste.
Tabelle dei componenti e vincoli strutturali	Dati globali in `config.py`	Comprendono codici, valenze, diametri, co-occorrenze, triplette e coppie vietate; non sono impostazioni operative del singolo run.

4.2 Elaborazione di un singolo nodo

Per validare un nuovo dataset si consiglia di iniziare da un ID con viste chiare. Per il nodo `7`:

```
conda activate graph_rec
python run.py 7
```

`run.py` verifica che l'ID esista tra le immagini scoperte e passa le viste del nodo all'orchestratore. Se l'ID non viene trovato, stampa il percorso di `IMAGES_DIR` che ha controllato: è un'indicazione utile per capire se il problema è nel file `.env` o nei nomi delle immagini.

L'exit code non è sufficiente a certificare il run. `run.py` può terminare con codice zero anche se l'ID non è stato trovato o se nessuna vista ha prodotto detection valide. Cerca nel log l'intestazione `NODE 7`, il blocco `FINAL GRAPH` e il percorso dell'output, poi verifica che `fused_graph.json` sia stato appena scritto.

La prima elaborazione può richiedere più tempo per la generazione dei file VGGT. Prima di interromperla, verificare l'attività di CPU, GPU o disco e la creazione degli artefatti nella cartella del nodo.

Interpretazione del log

Il log articola la lavorazione in tre step principali. Nello *Step 1* ogni immagine viene aperta, analizzata da YOLO e trasformata in un insieme di osservazioni valide. Una riga come `node7_0002.jpg -> 5 nodes` significa che, dopo i filtri, quella vista ha prodotto cinque componenti utilizzabili.

Prima dello *Step 2* possono comparire i messaggi VGGT. Se gli artefatti `.npz` mancano, l'interprete indicato da `VGGT_EMBEDDED_PYTHON` li genera; se sono già presenti, li riusa. Lo Step 2 è la ReID: raggruppa le osservazioni delle varie fotografie in *scene node*, cioè nelle istanze fisiche che la pipeline ritiene presenti nella scena.

Lo *Step 3* costruisce gli archi a partire dagli indizi geometrici, prova a collegare le componenti rimaste separate e associa i diametri. Il riepilogo `FINAL GRAPH` indica quanti nodi e archi sono stati ottenuti, quanti nodi sono isolati e se il grafo finale è connesso.

In fondo viene stampato il percorso della cartella di output. È il riferimento più affidabile quando `WORK_DIR` non coincide con `./output`.

4.3 Output prodotti

Con la configurazione del Capitolo 3, il nodo `7` viene scritto in:

`D:/GraphReconstructionData/work/impianto_A/multiview/node7/`

La struttura tipica, dopo ricostruzione e controllo visivo, è questa:

```
work/impianto_A/
  detection_cache/
    node7_0001.json
    node7_0002.json
  multiview/
    node7/
```

```
    fused_graph.json
    scene_nodes.json
    bird_view.png
    vggt_geometry.json
    vggt/
      generation_request.json
      generation_summary.json
      summary.json
      node7_0001.npz
      node7_0002.npz
    fused_graph_plot.png
    fused_graph_qgis_combined.geojson
    standard_similarity.json
standard_comparison/
  similarity_table.csv
  similarity_table.json
  similarity_table.html
  best_matches.csv
  report_completo.html
```

Non tutti i file compaiono con il solo `run.py`. `fused_graph_plot.png`, il GeoJSON e i risultati di confronto vengono aggiunti dagli entrypoint di visualizzazione e valutazione. Inoltre VGGT può essere disabilitato o escluso per alcuni nodi; in quel caso i relativi file non sono un requisito di successo.

Output principale: `fused_graph.json`

Questo file è destinato alle applicazioni a valle. Riporta il numero totale di immagini, di nodi e di archi e indica se il grafo è connesso. Seguono due liste, `nodes` ed `edges`.

Ogni elemento di `nodes` rappresenta un componente fisico. Oltre a codice e nome, conserva la confidenza, le immagini che lo supportano, il grado nel grafo e la valenza massima prevista. `support_count` è particolarmente utile nel controllo: un componente osservato in più viste è in genere meglio supportato di un componente derivato da una sola detection.

Gli elementi di `edges` descrivono le connessioni. I campi `vggt_score`, `vggt_support_count` ed `evidence_sources` spiegano quale evidenza ha sostenuto l'arco. Se `forced` vale `true`, il collegamento è stato aggiunto dal refinement per rendere il grafo connesso. Non è necessariamente sbagliato, ma merita più attenzione nel controllo umano, soprattutto quando `forced_reason` indica un fallback strutturale.

I diametri compaiono come `diameter_mm`. I campi di confidenza e conflitto permettono di distinguere un valore sostenuto in modo coerente da un ramo sul quale sono state osservate indicazioni discordanti.

Tracciabilità ReID: `scene_nodes.json`

`scene_nodes.json` è il ponte tra fotografie e grafo. Per ogni istanza fisica riporta le immagini di supporto e gli identificativi delle osservazioni che sono state fuse. Se nel grafo finale compaiono due componenti che sembrano lo stesso oggetto, questo è il primo file da aprire: permette di vedere se la ReID ha lasciato separate detection provenienti da viste diverse.

È un artefatto di tracciabilità, normalmente escluso dalle interfacce applicative.

Geometria e render di controllo

I file nella directory `vggt/` sono dati numerici intermedi costosi da generare. `vggt_geometry.json` raccoglie invece la geometria aggregata, gli indizi di connessione e le eventuali correzioni di codice riutilizzabili nei run successivi.

`bird_view.png` viene creato durante la ricostruzione quando esiste geometria sufficiente per almeno due scene node. La sua assenza non indica da sola un fallimento, soprattutto se VGGT è disabilitato. Quando esiste, offre una vista rapida della geometria ma non sostituisce il render del grafo. Per ottenere quest'ultimo eseguire:

```
python visualize.py 7
```

Il comando legge `fused_graph.json`, salva `fused_graph_plot.png`, crea un GeoJSON con layout di visualizzazione e prova a generare il report con lo standard più simile.

Nel PNG verificare prima numero e tipo dei componenti, quindi gli archi. Se i componenti sono già duplicati o mancanti, il problema precede la costruzione del grafo ed è probabilmente nella detection o nella ReID.

4.4 Verifica del risultato

Valutazione con ground truth

Se sono disponibili label estese e grafo annotato, eseguire:

```
python evaluate.py 7
```

Il terminale mostra true positive, false positive, false negative, precision, recall, F1 e purity delle istanze. Confronta inoltre il grafo con gli standard di riferimento e salva la tabella completa sotto `<WORK_DIR>/standard_comparison/`.

Le metriche su istanze e archi vengono stampate nel terminale ma non sono salvate automaticamente in un report dedicato. Per una lavorazione tracciabile è quindi opportuno acquisire anche il log completo del comando. I file persistiti da questa fase riguardano soprattutto il confronto con gli standard.

Se mancano le label, la parte supervisionata viene saltata. Il confronto strutturale con gli standard può comunque essere eseguito sul grafo ricostruito, perché non dipende dalle bounding box ground truth.

I report aggregati sotto `standard_comparison/` descrivono gli ID passati nell'esecuzione corrente. Un comando limitato a un solo nodo può quindi sostituire un report creato in precedenza per l'intero batch. Genera i report di consegna soltanto dopo aver stabilito l'insieme definitivo di nodi.

4.5 Esecuzione batch e preparazione della consegna

Elaborazione batch

Dopo la validazione del nodo campione, `run.py` senza argomenti elabora gli ID rilevati che non appartengono a `SKIP_NODES` e che hanno almeno il numero di viste definito da `MIN_IMAGES`;

entrambi i valori si trovano in `graph_reconstruction/config.py`:

```
python run.py
```

Con Conda si raccomanda di eseguire i tre entrypoint nello stesso terminale:

```
python run.py
python evaluate.py
python visualize.py --all
```

Per la sola inferenza, senza dati annotati, è disponibile l'entrypoint semplificato che esegue anche l'export QGIS:

```
python infer.py D:/percorso/immagini
```

L'export usa automaticamente la geometria VGGT quando disponibile; in caso contrario applica il layout CAD.

Per limitare il flusso a un sottoinsieme, indicare gli stessi ID nei tre comandi:

```
python run.py 2 7 15_1
python evaluate.py 2 7 15_1
python visualize.py 2 7 15_1
```

Export per QGIS

Per convertire tutti i grafi disponibili:

```
python -m graph_reconstruction.qgis
```

Per scegliere la geometria disponibile su un solo nodo:

```
python -m graph_reconstruction.qgis `
  D:/GraphReconstructionData/work/impianto_A/multiview/node7/fused_graph.json `
  --geometry-source auto
```

Il GeoJSON mantiene nodi, archi e attributi, ma le coordinate sono locali alla scena oppure sintetiche. Prima di sovrapporlo a cartografia reale occorre una georeferenziazione esterna.

`evaluate.py` e `visualize.py` generano entrambi un GeoJSON con geometria `fake` e possono sovrascrivere un export creato in precedenza con `--geometry-source auto` o `vggt`. Se il GeoJSON fa parte della consegna, esegui l'export con la geometria desiderata come ultimo passaggio del flusso.

Cache e riesecuzioni

La cache delle detection si trova in `<WORK_DIR>/detection_cache/`; i file VGGT restano nella cartella del nodo. Questa scelta rende i run successivi più veloci, ma richiede disciplina: la cache di detection non riconosce ogni possibile cambio dell'immagine, del modello o del decoder.

Quando cambia il dataset o il modello YOLO, utilizzare una nuova `WORK_DIR`. Questa scelta evita cancellazioni selettive e permette di confrontare le due lavorazioni senza perdere traccia di quale cache sia stata usata.

Checklist prima del batch completo

Prima di avviare un batch completo verifica che:

- l'ambiente Conda corretto sia attivo e il decoder sia importabile;
- `.env` punti al dataset, ai pesi e alla directory di lavoro della commessa corrente;
- i nomi delle immagini rispettino il pattern e un singolo ID venga scoperto correttamente;
- il volume di `WORK_DIR` abbia spazio per detection, point map e report;
- il nodo campione produca `fused_graph.json` e un render leggibile;
- gli archi forzati del campione siano stati controllati;
- le metriche siano disponibili oppure sia registrato il motivo per cui il ground truth manca;
- versione del codice, configurazione e identificativo dei pesi YOLO vengano conservati insieme agli output;
- dati vecchi e nuovi non condividano la stessa directory di lavoro.

Una consegna minima comprende `fused_graph.json` e il relativo `fused_graph_plot.png`. A seconda del processo possono essere aggiunti il report di valutazione, `scene_nodes.json` per la tracciabilità e il GeoJSON per il passaggio a QGIS. Prima dell'invio esterno, verificare i metadati: request VGGT, summary e GeoJSON possono contenere percorsi assoluti della macchina di elaborazione. Cache detection e file `.npz` non fanno normalmente parte della consegna.

Parte II

Architettura e riferimento tecnico

Capitolo 5

Repository ed entrypoint

5.1 Punti di ingresso

Per l'uso operativo, `infer.py` riceve direttamente la cartella delle immagini, ricostruisce tutti i nodi trovati ed esporta il risultato per QGIS:

```
python infer.py D:/percorso/immagini
```

Non richiede label o annotazioni. L'export usa la geometria VGGT quando disponibile e ripiega automaticamente sul layout CAD. `run.py` rimane l'entrypoint tecnico per avviare la sola ricostruzione usando la configurazione del progetto:

```
python run.py
```

Senza argomenti vengono elaborati tutti i nodi ammessi dalla configurazione; gli eventuali ID servono soltanto a limitare la lavorazione, come descritto nel Capitolo 4. `run.py` mantiene volutamente un ruolo ridotto: interpreta la richiesta e delega la ricostruzione a `run_multiview_all()` o `run_multiview_pipeline()`.

Valutazione, visualizzazione aggiuntiva ed export QGIS sono operazioni separate e non sono necessarie per ottenere il grafo ricostruito.

5.2 Struttura del repository

La tabella riporta i percorsi utili per orientarsi nel codice, senza elencare ogni file di servizio presente nel repository.

Elemento	Tipo o origine	Descrizione
`infer.py`	Entrypoint operativo	Riceve le immagini, avvia la ricostruzione ed esporta il GeoJSON per QGIS.
`run.py`	Entrypoint tecnico	Avvia la sola ricostruzione completa o quella dei soli ID indicati.
`graph_reconstruction/config.py`	Configurazione	Carica `.env` e definisce percorsi, soglie e vincoli globali.
`graph_reconstruction/pipeline/`	Pipeline	Contiene l'orchestrazione e le fasi di detection, ReID, supporto VGGT e costruzione del grafo.

Elemento	Tipo o origine	Descrizione
`graph_reconstruction/shared/`, `graph_reconstruction/visualization/`	Moduli condivisi	Raccolgono funzioni di supporto, accesso ai dati e visualizzazioni usate dalla pipeline.
`models/`, `graph_reconstruction/assets/`, `data/`	Risorse	Ospitano checkpoint, primitive grafiche e la struttura di immagini, label e annotazioni; i percorsi effettivi sono stabiliti dalla configurazione.
`output/`	Artefatti	Destinazione predefinita di risultati, cache e file diagnostici.

Architettura funzionale

La pipeline è prevalentemente funzionale: l'orchestratore coordina funzioni che trasformano dataclass, dizionari e grafi NetworkX. Il flusso di chiamata è descritto nel Capitolo 6, mentre gli oggetti scambiati tra le fasi sono raccolti nel Capitolo 7.

5.3 Strumenti successivi alla ricostruzione

Gli script seguenti leggono gli artefatti già prodotti dalla pipeline. Sono utili per analisi o consegne specifiche, ma non fanno parte dell'esecuzione necessaria a ottenere il grafo.

Valutazione

`evaluate.py` usa `scene_nodes.json`, `fused_graph.json`, la cache delle detection e, quando disponibili, le label estese e le annotazioni di riferimento. Calcola le metriche sulle istanze e sugli archi e avvia il confronto con gli standard. L'assenza dei dati di riferimento limita la valutazione, non la ricostruzione.

Visualizzazione ed export QGIS

`visualize.py` legge `fused_graph.json` e produce `fused_graph_plot.png`; può inoltre generare un GeoJSON sintetico e il report del best match. `graph_reconstruction/qgis.py` gestisce invece l'export QGIS dedicato, per un singolo grafo o per tutti quelli disponibili, usando geometria VGGT quando richiesta e disponibile oppure coordinate sintetiche. Nessuno dei due modifica il JSON ricostruito.

Diagnostica e manutenzione

Le utility in `tools/debug/` servono per ispezioni mirate. Gli script in `tools/benchmark/` controllano invece requisiti e consumo di risorse. Non sono dipendenze operative del flusso normale.

5.4 Caricamento della configurazione

`graph_reconstruction/config.py` viene importato prima dell'avvio della pipeline. In questa fase legge il file `.env` nella root senza sovrascrivere eventuali valori già presenti nell'ambiente di processo: una variabile impostata dall'esterno ha quindi la precedenza.

La funzione `_path_setting()` mantiene i percorsi assoluti e risolve quelli relativi rispetto alla root del progetto. Se manca un percorso obbligatorio, solleva un `RuntimeError` durante l'import, prima di `main()`. La mappa delle impostazioni operative e dei parametri globali è riportata nel Capitolo 4 e non viene duplicata qui.

Capitolo 6

Flusso di elaborazione nel codice

6.1 Orchestrazione e sequenza generale

`run.py` seleziona gli ID richiesti e delega ogni lavorazione a `run_multiview_pipeline()` in `pipeline/orchestrator.py`. Da questo punto l'orchestratore coordina l'intera trasformazione, senza contenere direttamente gli algoritmi delle singole fasi.

Per ciascun nodo vengono costruite prima le osservazioni delle singole viste. Gli artefatti VGGT, se abilitati, forniscono la geometria usata dalla ReID e dalla successiva stima delle connessioni. Le istanze riconciliate diventano quindi nodi di un grafo inizialmente privo di cicli; il refinement collega le componenti rimaste separate, prima dell'annotazione dei diametri e della serializzazione finale. La Figura 6.1 riassume le dipendenze tra queste fasi.

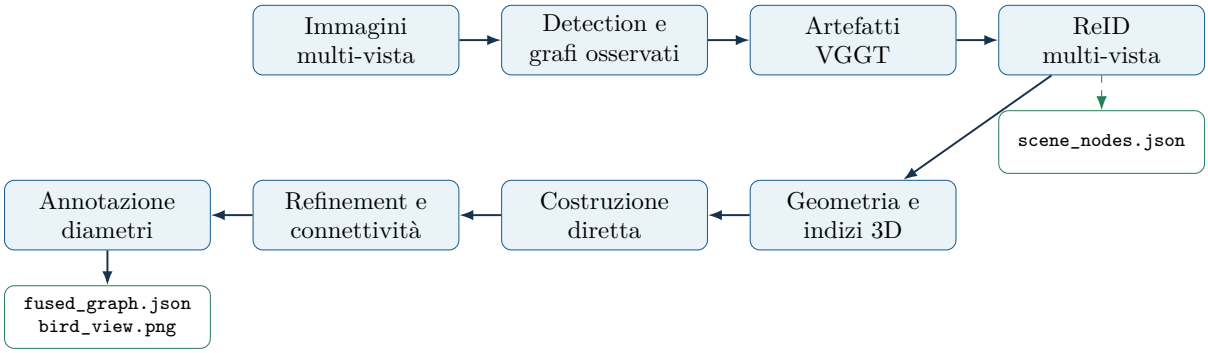


Figura 6.1: Le trasformazioni coordinate da `orchestrator.py`.

6.2 Detection e riconciliazione delle osservazioni

Detection e grafi osservati

`build_all_observed_graphs()` analizza ogni immagine tramite `detect_and_decode_cached()`. Le detection sotto la soglia vengono scartate, mentre i codici di diametro sono separati dai componenti topologici. Ogni detection restante diventa un `ObservedNode` e le osservazioni della stessa immagine vengono raccolte in un `ObservedLocalGraph`.

In questa fase non vengono stimati archi locali. La suddivisione per vista conserva invece la provenienza delle osservazioni e introduce il primo vincolo della ReID: due detection presenti nella stessa immagine non possono rappresentare la stessa istanza fisica.

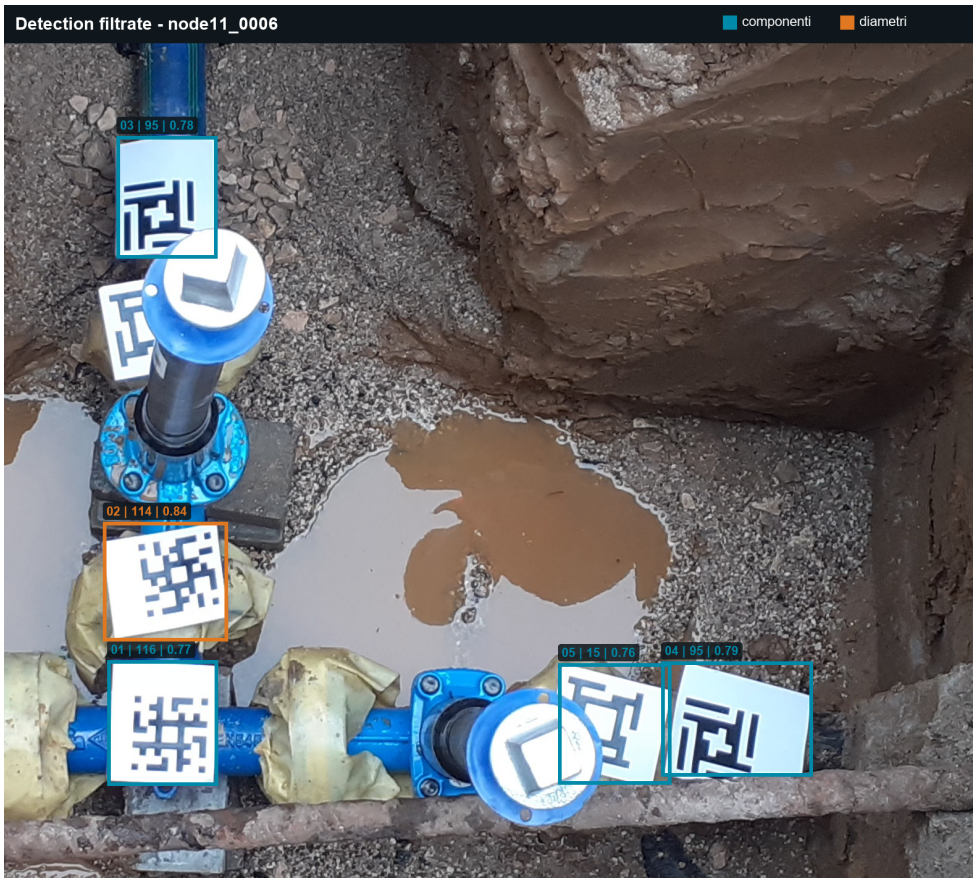


Figura 6.2: Detection filtrate in una vista del nodo 11

Costruzione degli `SceneNode`

`identify_scene_nodes()` in `node_reid.py` raggruppa prima le osservazioni per codice. Per ogni codice costruisce un grafo di incompatibilità: le osservazioni della stessa fotografia devono restare in gruppi diversi. La colorazione greedy fornisce così un limite inferiore al numero di istanze fisiche necessario a spiegare le co-occorrenze.

Quando la geometria è disponibile, il clustering agglomerativo parte dalle singole osservazioni e fonde iterativamente la coppia compatibile migliore in base alla distanza 3D tra le point cloud, senza unire detection della stessa immagine. Il codice prevede anche un segnale basato sui vicinati osservati, ma nella baseline corrente questo contributo resta inattivo per il limite descritto nel Capitolo 7. Senza geometria viene usata direttamente l'assegnazione ottenuta dalla colorazione.

Ogni gruppo risultante diventa un `SceneNode`; l'associazione con le osservazioni originali viene salvata in `scene_nodes.json`.



Figura 6.3: Osservazioni raggruppate dalla ReID

6.3 Geometria VGGT e costruzione degli indizi

Artefatti e geometria delle osservazioni

Se VGGT è attivo per il nodo, l'orchestratore chiama `ensure_vggt_artifacts()`. Per ogni vista viene riutilizzata la point map già disponibile oppure, se consentito, viene generato il corrispondente file `.npz`. All'interno delle bounding box vengono quindi campionati i punti 3D usati per stimare la geometria delle singole osservazioni.

La stessa geometria interviene in due momenti: supporta la riconciliazione delle osservazioni durante la ReID e viene poi aggregata per `SceneNode` per valutare le possibili connessioni.

Indizi per le connessioni

`build_vggt_edge_hints()` confronta le coppie di `SceneNode` co-visibili. Per ciascuna raccoglie le viste di supporto e combina distanza tra le point cloud o le porte, allineamento degli assi e ranking reciproco tra vicini. Le coppie vietate vengono escluse, mentre le co-occorrenze note possono rafforzare il punteggio.

Il risultato è ancora un insieme di *hint*, non un insieme di archi. La geometria aggregata e gli indizi vengono salvati in `vggt_geometry.json` e possono essere riutilizzati quando la cache è compatibile con gli `SceneNode` del run corrente.

6.4 Costruzione, completamento e annotazione del grafo

Costruzione diretta

Dopo il tentativo di riassorbire eventuali singleton spuri, `build_direct_graph()` crea un `networkx.Graph` con un nodo per ogni `SceneNode`. Le coppie suggerite da VGGT vengono ordinate per numero di voti e punteggio e valutate in ordine greedy.

Una coppia viene accettata soltanto se non è vietata, se rispetta la valenza disponibile dei componenti e se non introduce un ciclo. Il risultato iniziale è quindi una foresta. Le coppie esaminate restano disponibili come `BridgeEdge` per il completamento successivo.

Completamento, diametri e output

Se la foresta non è connessa, `force_spanning_tree()` prova prima i `BridgeEdge` che uniscono componenti diverse. Se non bastano, applica un fallback strutturale basato su co-occorrenze, triplette e supporto delle istanze. Gli archi aggiunti in questa fase sono riconoscibili dall'attributo `forced=true`.

`annotate_graph_diameters()` associa ogni codice diametro all'arco visibile più vicino e propaga il valore lungo il gruppo di archi contiguo. La propagazione si interrompe sui componenti configurati come possibili cambi di diametro.

Infine `serialize_fused_graph()` salva struttura ed evidenze in `fused_graph.json`; `render_bird_view()` produce la vista d'insieme `bird_view.png`.

Capitolo 7

Modello dati e mappa del codice

Il Capitolo 6 descrive l'ordine delle elaborazioni. Qui sono raccolti soltanto gli oggetti scambiati tra le fasi e i moduli nei quali intervenire. Le procedure di diagnosi e gli strumenti accessori sono invece nel Capitolo 8.

7.1 Rappresentazioni principali

Durante un run lo stesso componente passa da osservazione in una fotografia a istanza fisica riconciliata, quindi a nodo del grafo. Quando si analizza un'anomalia conviene individuare la prima rappresentazione nella quale il dato non corrisponde più a quello atteso.

Elemento	Tipo o origine	Descrizione
`ObservedNode`	Dataclass in `pipeline/observed_graphs.py`	Rappresenta una detection valida in una singola immagine. Conserva codice, confidenza, bounding box, centro 2D e identificativo della vista.
`ObservedLocalGraph`	Dataclass in `pipeline/observed_graphs.py`	Raggruppa gli `ObservedNode` di una vista. Nella baseline corrente `edges` e `local_patterns` restano vuoti: non contiene una topologia 2D stimata.
`SceneNode`	Dataclass in `pipeline/node_reid.py`	Rappresenta un componente fisico ottenuto riconciliando osservazioni di viste diverse. Mantiene identità fusa, confidenza, osservazioni di supporto e valenza massima. Viene serializzato in `scene_nodes.json`.
`BridgeEdge`	Dataclass in `pipeline/direct_graph_builder.py`	Descrive una connessione candidata con evidenze geometriche e viste di supporto. È un record distinto dall'arco NetworkX; l'elenco dei candidati viene riutilizzato dal refinement per collegare componenti separate.
`networkx.Graph`	Struttura in memoria	È il grafo ricostruito. I nodi derivano dagli `SceneNode`; gli archi conservano evidenze, forcing ed eventuale diametro. La serializzazione produce `fused_graph.json`.

7.2 Moduli da conoscere

La tabella individua i punti di intervento della pipeline senza ripetere gli algoritmi già descritti nel capitolo precedente.

Elemento	Tipo o origine	Descrizione
`shared/detection.py`	Acquisizione	Scopre gli ID disponibili, carica YOLO e il decoder e gestisce la cache delle detection per immagine.
`pipeline/orchestrator.py`	Coordinamento	Implementa `run_multiview_pipeline()` e `run_multiview_all()`; ordina le fasi, crea le directory e salva gli artefatti.
`pipeline/observed_graphs.py`	Osservazioni	Filtra le detection, esclude i codici diametro dalla topologia e costruisce gli oggetti per vista.
`pipeline/node_reid.py`	ReID	Raggruppa le osservazioni in `SceneNode`, costruisce la mappa osservazione–istanza e prova a riassorbire i singleton spuri.
`pipeline/vggt_runtime.py`, `pipeline/vggt_export_cli.py`	Runtime VGGT	Controllano gli artefatti disponibili e avviano, tramite l'interprete configurato, la generazione delle point map mancanti.
`pipeline/vggt_3d_edges.py`	Geometria	Stima la geometria delle osservazioni, la aggrega per `SceneNode` e produce gli hint usati per valutare le connessioni.
`pipeline/direct_graph_builder.py`	Grafo iniziale	Costruisce la foresta rispettando coppie vietate, valenze e assenza di cicli; restituisce anche i bridge candidati.
`pipeline/graph_refinement.py`	Completamento e output	Collega le componenti rimaste separate, marca gli archi forzati e serializza il grafo finale.
`pipeline/diameter_annotation.py`	Diametri	Associa le detection di diametro agli archi visibili e propaga il valore lungo i rami compatibili.

La configurazione operativa e i parametri globali sono riepilogati nel Capitolo 4. Valutazione, visualizzazione e strumenti di debug non appartengono alla pipeline principale e sono raccolti nel Capitolo 8.

7.3 Limiti implementativi noti

Vicinato strutturale ReID.
Il vicinato viene attualmente ricavato da una collezione già limitata alle osservazioni dello stesso codice. Dopo l'esclusione del codice stesso, il vicinato risulta vuoto; la selezione delle fusioni dipende quindi soprattutto dal vincolo sulla stessa immagine e dalla distanza 3D.
Correzioni codice VGGT.
Nei rami di `vggt_3d_edges.py` che applicano o riproducono una correzione, `COMPONENT_INFO` viene trattato come un dizionario con campo `name`, mentre in `config.py` restituisce una tupla `(valenza, nome)`. Se una correzione cambia realmente il codice, questo ramo può sollevare un `AttributeError`.

Questi limiti non bloccano i casi che non attivano i relativi rami, ma devono essere coperti da casi di regressione prima di intervenire su ReID o correzioni VGGT.

Capitolo 8

Diagnosi e verifica delle modifiche

Questo capitolo serve a localizzare un errore e a verificare una modifica. I comandi ordinari di esecuzione e valutazione restano nel Capitolo 4; qui sono riportati soltanto i controlli utili quando un risultato non è quello atteso.

8.1 Diagnosi di un nodo

La regola principale è cercare il primo artefatto errato: detection, riconciliazione delle istanze, geometria, archi e infine diametri. Per conservare lo stato intermedio, abilitare temporaneamente i dump in `.env`:

```
DEBUG_OUTPUT_ENABLED=true
DEBUG_OUTPUT_DIR=./output/debug
```

Rieseguire quindi un solo nodo:

```
python run.py 7
```

I file interni vengono scritti sotto `<DEBUG_OUTPUT_DIR>/node7/`. Non fanno parte della consegna e gli strumenti che li leggono non sono eseguiti automaticamente da `run.py`. Al termine riportare `DEBUG_OUTPUT_ENABLED` a `false`.

Elemento	Tipo o origine	Descrizione
Componente assente	`<WORK_DIR>/detection_cache/`	Verificare che il codice compaia nella cache della relativa immagine. Se manca, il problema precede la ReID ed è da cercare tra YOLO, crop e decoder.
Componente duplicato	`<WORK_DIR>/multiview/node<id>/scene_nodes.json`	Confrontare le immagini di supporto delle istanze con lo stesso codice. Supporti complementari indicano una possibile mancata fusione.
Separazione ReID errata	`<DEBUG_OUTPUT_DIR>/node<id>/reid_geometry.json`	Controllare posizione 3D, appartenenza dei crop e vincoli tra osservazioni prima di modificare il clustering.
Arco mancante o errato	`<WORK_DIR>/multiview/node<id>/`	Verificare prima gli `SceneNode`, quindi `vggt_geometry.json` e infine gli archi in `fused_graph.json`.

Elemento	Tipo o origine	Descrizione
Diametro incoerente	`<DEBUG_OUTPUT_DIR>/node<id>/diameters.json`	Confrontare osservazioni, associazione iniziale e propagazione. I codici `250` e `252` non hanno un valore millimetrico nella mappa corrente.

I comandi diagnostici più utili sono:

```
python tools/debug/debug_crops.py 7
python tools/debug/analyze_reid_false_positives.py 7
python tools/debug/plot_reid_geometry.py 7 15 95
python tools/debug/debug_graph_projection.py 7
```

Il primo crea una tavola dei crop per `SceneNode`; il secondo, se sono disponibili label e annotazioni, confronta i cluster con il ground truth; il terzo visualizza la separazione 3D per i codici richiesti; l'ultimo proietta nodi e archi sulle immagini e salva un `consistency_report.json`. Le distanze 2D mostrate da quest'ultimo sono un aiuto visivo e non un'evidenza usata per costruire il grafo.

ID composti. Tra i comandi elencati, `debug_crops.py` e `debug_graph_projection.py` convertono l'argomento in numero intero e non accettano identificativi come `15_1`. Per questi nodi occorre adattare il parsing oppure richiamare la funzione interna passando l'ID come stringa.

8.2 Verifica delle modifiche

Una modifica alla pipeline va confrontata sugli stessi nodi e sugli stessi input. Quando cambiano modello o dataset è preferibile usare una nuova `WORK_DIR`, così cache e risultati precedenti restano disponibili.

La verifica minima consiste nel:

1. rieseguire almeno un nodo che attraversi il ramo modificato;
2. confrontare il primo artefatto interessato e il `fused_graph.json` finale;
3. eseguire valutazione e visualizzazione come descritto nel Capitolo 4;
4. conservare comando, configurazione e differenze osservate.

L'edge F1 di `evaluate.py` usa il miglior abbinamento biunivoco trovato tra istanze predette e annotate, tenendo conto anche della struttura. Va quindi letto come misura della qualità del grafo sotto quel mapping, non come verifica della stabilità degli identificativi prodotti.

Il repository non contiene una suite di test dedicata. Prima di rifattorizzare ReID, costruzione degli archi o diametri è opportuno congelare almeno un caso per ciascuno di questi comportamenti: fusione multi-vista con VGGT, fallback di connessione e propagazione del diametro.

Capitolo A

Riferimento operativo

I comandi riportati in questa appendice devono essere eseguiti dalla root del repository.

A.1 Ambiente e verifica

Comando	Uso
`conda env create -f environment.yml`	Crea l'ambiente principale dal file versionato.
`conda activate graph_rec`	Attiva l'ambiente prima degli entrypoint.
`conda env update -n graph_rec -f environment.yml --prune`	Riallinea l'ambiente da `environment.yml`.
`python -c "from graph_reconstruction.config import IMAGES_DIR, WORK_DIR; print(IMAGES_DIR, WORK_DIR)"`	Verifica i percorsi caricati da `.env`.

A.2 Comandi principali

Comando	Risultato
`python infer.py D:/percorso/immagini`	Esegue ricostruzione ed export QGIS senza richiedere annotazioni.
`python run.py 7`	Ricostruisce soltanto il nodo 7.
`python run.py 2 7 15_1`	Ricostruisce gli ID indicati.
`python run.py`	Ricostruisce tutti gli ID scoperti.
`python evaluate.py 7`	Stampa le metriche del nodo e aggiorna il confronto con gli standard.
`python evaluate.py`	Valuta tutti i risultati presenti.
`python visualize.py 7`	Genera il plot, il GeoJSON sintetico e il report visuale del nodo.
`python visualize.py --all`	Visualizza tutti i grafi disponibili.

Comando	Risultato
`python -m graph_reconstruction.qgis`	Esporta in batch i grafi disponibili. Eseguire per ultimo se serve una geometria non `fake`.
`python -m graph_reconstruction.visualization.shurikode_symbols --all`	Genera i render SVG con i simboli Shurikode.

A.3 Percorsi e artefatti

Percorso	Contenuto
`<IMAGES_DIR>/node<id>_<frame>.jpg`	Immagini della lavorazione; la ricerca non è ricorsiva.
`<LABELS_EXT_DIR>/<image_stem>_extended.txt`	Label estese usate dalla valutazione.
`<ANNOTATIONS_DIR>/node<id>_gt.json`	Ground truth del grafo.
`<WORK_DIR>/detection_cache/<image_stem>.json`	Detection e decoding memorizzati per immagine.
`<WORK_DIR>/multiview/node<id>/fused_graph.json`	Risultato principale.
`<WORK_DIR>/multiview/node<id>/scene_nodes.json`	Tracciabilità della ReID.
`<WORK_DIR>/multiview/node<id>/vggt/`	Artefatti geometrici per vista.
`<WORK_DIR>/multiview/node<id>/vggt_geometry.json`	Geometria aggregata, hint e cache VGGT.
`<WORK_DIR>/multiview/node<id>/fused_graph_plot.png`	Render prodotto da `visualize.py`.
`<WORK_DIR>/multiview/node<id>/standard_similarity.json`	Ranking del nodo rispetto agli standard.
`<WORK_DIR>/standard_comparison/`	Tabelle e report aggregati.
`<DEBUG_OUTPUT_DIR>/node<id>/`	Dump interni presenti soltanto con debug attivo.

A.4 Variabili di `.env`

Variabile	Default	Significato
`IMAGES_DIR`	`./data/images`	Cartella immediata delle immagini.
`LABELS_EXT_DIR`	`./data/labels_extended`	Label estese per la valutazione.

Variabile	Default	Significato
YOLO_MODEL_PATH	./models/yolo_detector.pt	File dei pesi YOLO.
ASSETS_DIR	./graph_reconstruction/assets	Mapping, porte e SVG Shurikode.
ANNOTATIONS_DIR	./data/graph_annotations	Grafi ground truth.
WORK_DIR	./output	Root di risultati e cache.
DEBUG_OUTPUT_ENABLED	false	Abilita i dump interni.
DEBUG_OUTPUT_DIR	./output/debug	Root del debug.
VGGT_EDGE_ENABLED	true	Abilita geometria e hint VGGT.
VGGT_GENERATE_IF_MISSING	true	Genera i file .npz mancanti.
VGGT_EMBEDDED_PYTHON	interprete attivo	Runtime usato da VGGT; può essere sovrascritto.
VGGT_MODEL_ID	facebook/VGGT-1B	Identificativo del modello.
VGGT_PREPROCESS_MODE	crop	Pre-processing delle immagini VGGT.

A.5 Verifica minima del risultato

Prima di considerare concluso un nodo:

- verificare che fused_graph.json sia stato aggiornato dal run;
- controllare is_connected, il numero di nodi e il numero di archi;
- individuare archi con forced=true e valori diameter_conflict=true;
- aprire fused_graph_plot.png;
- conservare il log della valutazione;
- eseguire per ultimo l'eventuale export QGIS definitivo.